SCALE FOR PROJECT PYTHON MODULE 10 (/PROJECTS/PYTHON-MODULE-10)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42madrid.com:vogsphere/intra-uuid-01f95c04-b36e-49a
```

Introduction

- Remain polite, courteous, respectful and constructive
throughout the evaluation process. The well-being of the community
depends on it.

- Work with the person (or the group) being evaluated to identify any potential
dysfunctions in the work. Take the time to discuss
and debate the problems you have identified.

- You must consider that there may be differences in how your
peers have understood the project's instructions and the
scope of its functionalities. Always keep an open mind and grade
them as honestly as possible. The pedagogy is valid
only if peer evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student's or group's
Git repository.

- Double-check that the Git repository belongs to the student
or the group. Ensure that the work is for the relevant project
and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases were used to fool you
and make you evaluate something other than the content of the
official repository.

- To avoid any surprises, carefully check that both the evaluating
and the evaluated students have reviewed the possible scripts used
to facilitate the grading.

- If the evaluating student has not completed that particular
project yet, it is mandatory for this student to read the
entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository,
a non-functioning program, a norm error, cheating, etc. In these cases,
the grading is over and the final grade is 0 (or -42 in case of
cheating). However, with the exception of cheating, you are
encouraged to continue to discuss your work (even if you have not
finished it) in order to identify any issues that may have caused
this failure and avoid repeating the same mistake in the future.

- Remember that during the defence, no unexpected, premature, uncontrolled
termination of the program occurs; otherwise, the final grade is 0. Use the

appropriate flag.
Unless stated in the scale, you should never need to edit any file except the configuration file, if it exists.
If you want to edit a file, take the time to explain the reasons to the evaluated student and make sure both of you agree to this.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/203240/en.subject.pdf)

generator.tar.gz (https://cdn.intra.42.fr/document/document/48271/generator.tar.gz)

Preliminaries

Basics

- The review is done in the presence of the reviewed learner. This is how everyone progresses: by interacting with others.
    - No submitted work: 0, the review is over.
    - As soon as an exercise is non-functional, the review stops. You can look at the code of the following exercises, but they will not be graded.
- Check that the following files are present:
    - ex0/lambda_spells.py
    - ex1/higher_magic.py
    - ex2/scope_masteries.py
    - ex3/functools_artifacts.py
    - ex4/decorator_mastery.py If any required files are missing, the peer-review stops here.
- Verify that the code runs without errors and demonstrates the required concepts.
- Test both valid and invalid scenarios to ensure proper validation.
- Look for proper use of functionnal features: functionnal patterns, anonymous functions, lambda, decorators, functools...
- Ensure all dependcies are installed via proper package management in a virtual environment.
- Ensure the learner avoided Global variables and File I/O operations.

☑ Yes ☒ No

Exercises

Exercise 0 - Lambda Sanctum

Check that lambda_spells.py is properly implemented:

- Does artifact_sorter() use sorted() with a lambda to sort by 'power' descending?
- Does power_filter() use filter() with a lambda to filter mages by minimum power?
- Does spell_transformer() use map() with a lambda to add * prefix and suffix?
- Does mage_stats() use lambdas with max(), min() for statistics?

Test the functions with sample data:

Using data_generator.py in attachments to generate sample data:

- Create test artifacts and verify sorting works correctly
- Test power filtering with different minimum values
- Verify spell transformation adds the correct decorations to * spells *
- Check that statistics calculation returns correct max, min, and average

Verify that lambda expressions are used appropriately (not def functions for simple operations).
If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

Yes No

Exercise 1 - Higher Realm

Check that higher_magic.py demonstrates higher-order functions:

- Does spell_combiner() return a function that calls both input functions?
- Does power_amplifier() return a function that multiplies the base spell result?
- Does conditional_caster() return a function that checks condition before casting?
- Does spell_sequence() return a function that executes all spells in order?

Test the higher-order function behavior:

Using data_generator.py to have sample data for spells, targets and power values :

- Create simple test functions and verify combination works
- Test power amplification with different multipliers
- Verify conditional casting works when both true and false conditions are met
- Check that spell sequences execute all spells and return results list

Verify that functions are treated as first-class citizens (passed as arguments, returned from functions).
If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

Yes No

Exercise 2 - Memory Depths

Check that scope_mysteries.py demonstrates lexical scoping and closures:

- Does mage_counter() return a function that maintains count state?
- Does spell_accumulator() return a function that accumulates power over calls?
- Does enchantment_factory() return functions that remember their enchantment type?
- Does memory_vault() return a dict with store/recall functions sharing private storage?

Test closure behavior:

If you need to, generate sample data using generato_data.py

- Create multiple counters and verify they maintain separate state
- Test power accumulation with different initial values Example: create with intial_power=0. Call with 10, then 20, then 30.
- Create different enchantment factories and verify they work independently Example: create "Flaming" and "Frozen", and apply to same item "Axe". Expect different results.
- Test memory vault storage and recall functionality. Example: Store 3 values, recall all 3, recall 1 missing key. Add reviewee to explain what happems.

Verify that closures properly capture and maintain their lexical environment.
Verify there is no use of 'global' keyword, and no class definitions.
If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

Yes No

Exercise 3 - Ancient Library

Check that functools_artifacts.py properly uses the functools module:

Spell Reducer

- Does spell_reducer() use functools.reduce with operator module functions?
- Does spell_reducer() uses operator.add and operator.mul for "add" and "multiply"?
- Ask the learner to explain how are handled "min" and "max". Do you know other solutions?
- Test reduce with different operations (add, multiply, max, min)

Partial Enchanter

- Does partial_enchanter() use functools.partial to create specialized functions (not lambdas wrapping calls)?
- Verify partial application creates working specialized functions
- Test with any function(power, element, target) the result, with different powers

- Verify type(result['fire_enchant']) is functools.partial

Memoized Fibonacci

- Does memoized_fibonacci() use functools.lru_cache for memoization?
- Test fibonacci memoization improves performance on repeated calls. If you don't hnow how, ask the reviewee to show you how.
- Verify: memoized_fibonacci.cache_info() returns a CacheInfo with hits > 0 after calling fib(10) then fib(5)

Spell Dispatcher

- Does spell_dispatcher() use functools.singledispatch for type-based dispatch?
- Check single dispatch handles different types appropriately. Try also with a float.

Ask the learner

- "What happens if you call reduce on an empty list without an initializer?"
- "What is the difference between memoization and simply storing results in a global dict?"

If ALL tests pass and functools features are used (not manual reimplementation), tick "Yes." Otherwise, tick "No" and the peer-review is over.

Yes No

Exercise 4 - Master's Tower

As usual, you can generate sampls data for spells, names, Mages using data_generator.py

Check that decorator_mastery.py demonstrates decorator mastery:

- Does spell_timer() create a decorator that measures execution time?
- Does power_validator() create a parameterized decorator for validation?
- Does retry_spell() create a decorator that retries failed functions?
- Does MageGuild class properly use @staticmethod?
- Does cast_spell uses power_validator with min_power=10 ?

Test decorator functionality:

- Test spell_timer with a function and verify timing output
- Test power_validator with valid and invalid power levels
- Test retry_spell with functions that fail and eventually succeed
- Verify staticmethod works without instance
- Test MageGuild with valid and invalid names
- Test what cast_spells returns, on success and on failure. Try different powers.

Check that functools.wraps is used to preserve function metadata.
Verify that decorators properly transform function behavior.
If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

Yes No

Code Quality and Understanding

Code Quality and Understanding

Evaluate the learner's understanding of functional programming concepts:

- Can the learner explain the difference between lambda and def functions?
- Can the learner explain what makes functions "first-class citizens"?
- Does the learner understand how closures capture lexical scope?
- Can the learner explain the benefits of functools utilities?
- Does the learner understand how decorators transform functions?

Check for code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?

- Are type hints REQUIRED for all functions and methods (parameters and return values)?
- Docstrings are NOT required for this module.
- Is the code clean and readable?
- Are functional programming patterns used correctly?
- Does the code demonstrate understanding rather than just copying?

Discuss the concepts with the learner to assess their understanding.
If the learner demonstrates good understanding of functional programming, tick "Yes."

☑ Yes ☒ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work | Incomplete work | Norme | Cheat | Crash | Concerning situation

Leaks | Forbidden function | Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) | Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) | Privacy policy (https://profile.intra.42.fr/legal/terms/5) | General term of use of the site (https://profile.intra.42.fr/legal/terms/6) | Rules of procedure (https://profile.intra.42.fr/legal/terms/4) | Terms of use for video surveillance (https://profile.intra.42.fr/legal/terms/1) | Legal notices (https://profile.intra.42.fr/legal/terms/3)